ECS7032P - Statistical Planning and Reinforcement Learning

TAG: Tabletop Games Framework

Lab 1 - Getting started with TAG

What is TAG?

The Tabletop Games Framework (TAG) is a Java-based benchmark for developing modern board games for AI research. TAG provides a common skeleton for implementing tabletop games based on a common API for AI agents, a set of components and classes to easily add new games and an import module for defining data in JSON format. At present, this platform includes the implementation of several tabletop games that can also be used as an example for further developments. Additionally, TAG incorporates logging functionality that allows the user to perform a detailed analysis of the game, in terms of action space, branching factor, hidden information, and other measures of interest for Game AI research.

Getting started with TAG

Exercise 1 Follow the steps below to download, setup and run the Tabletop Games Framework. This assumes that you will install everything through IntelliJ as the Integrated Development Environment (IDE). In general this is the recommended option.

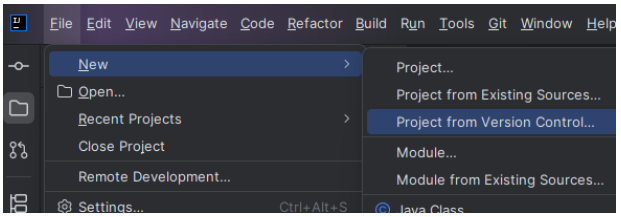
If you want to use some other IDE, such as Eclipse or VSCode, then you are welcome to do so but we will not be able to provide very much support in the labs. If you are familiar with git, then you can also clone the repository via the command line (`git clone https://github.com/GAIGResearch/TabletopGames`). If you prefer cloning the repository using a graphical user interface, you can try GitHub Desktop. There are also detailed instructions for cloning a repository at `https://docs.github.com/en/repositories/creating-and-managing-repositories/cloning-a-repository`.

TAG requires Java with minimum version 17. In order to get the framework installed and running, follow these instructions below.

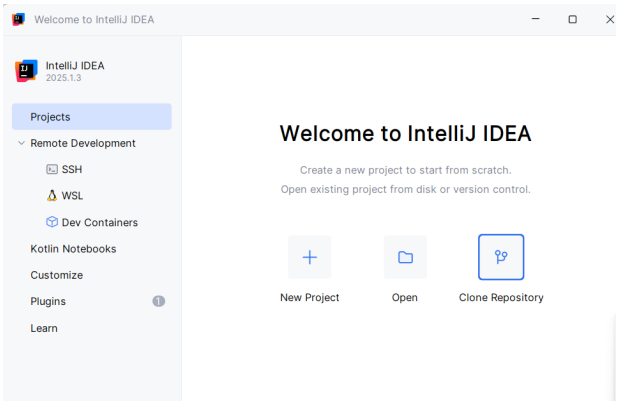
1. Install IntelliJ. There is a free version (the Community Edition) available from the JetBrains website at `https://www.jetbrains.com/idea/`. You can get free access to the Ultimate Edition if you want to with a few extra steps through your github account to evidence your student status. None of the extra features in the Ultimate Edition are needed for this course.

   Additionally IntelliJ is installed by default Queen Mary virtual machines accessible through the lab computers if you want to run this on the QM system. In this case, make sure that when you clone the repository you install it on your home drive, and not the local `C:` drive. One small downside here is that you cannot upgrade the IntelliJ version used.

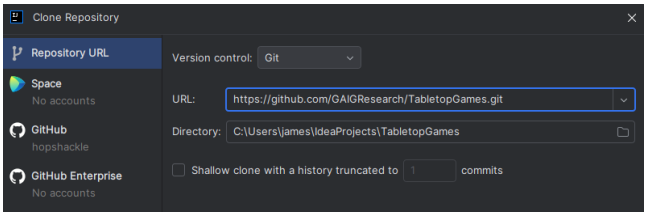
2. Clone the public github repository
   (a) Open IntelliJ IDEA
   (b) Select 'Create New Project' ... 'from version control'
      i. If you have a project already open, you'll find this option in the 'File' menu, 'New', etc.



ii. If you do not have any existing projects and instead see the Welcome to IntelliJ IDEA window (below), select the 'Clone Repository' option instead:



(c) In the new window, ensure that Version Control is set to Git, specify the directly to a location of your choice (the default is usually fine, but make sure this is not the local drive if using a QM machine), and specify the URL to clone as `https://github.com/GAIGResearch/TabletopGames.git`:



(d) Click on 'Clone'. You have now created a local clone of the public TAG repository.

3. Verify that TAG can execute in the IDE (the section on Troubleshooting with IntelliJ at the end of this document may be helpful if you have problems).:

   (a) In the IDE, open the file `src/main/java/core/Game.java` (you can navigate files on the left panel - click on *Project* if the folder hierarchy is not displayed).

   (b) With Game.java open and visible, click on menu `Run` → `Run 'Game.java'`, and click on the *Game* class in the pop up window. Alternatively, right click the file `Game.java` and select the option `Run 'Game.main()'`.

   (c) It'll take a few seconds for the project to be compiled and then it will run a game (likely 'Pandemic', but it may be a different one depending on the state of the repository).

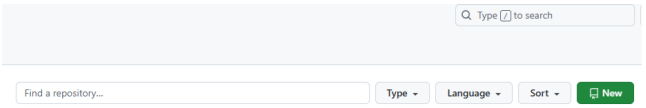
   (d) Then try the same thing by Running `src/main/java/gui/Frontend.java`. This will allow you to pick a game from a list of those implemented. Try Connect4, Poker or Tic-Tac-Toe.

4. Create a new cloud repository for your work. **[This is Optional.]**

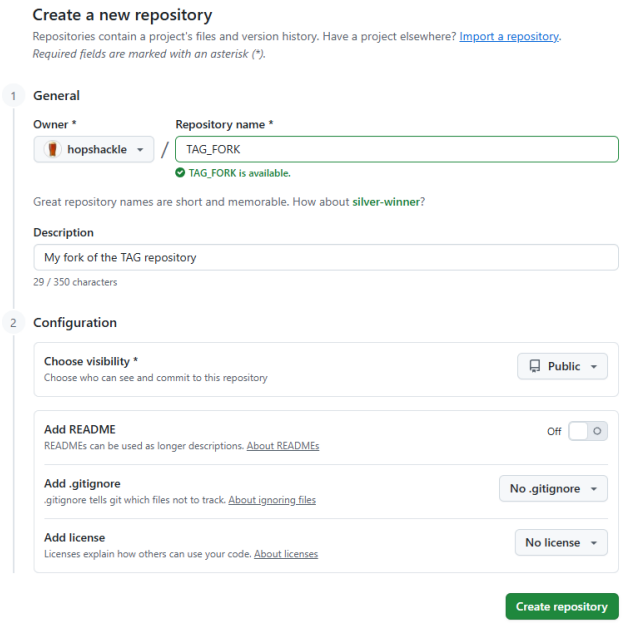
   Currently you have created a clone on your local machine. At some point, for example when working on the assigment later in the semester, you will probably want to have a version saved in the cloud for backup. The easiest way to do this is have a public github repository. This also makes it very easy to work as a group for the first assignment, as you can all commit to a single shared repository. There are many ways to do this, one relatively easy way using IntelliJ is described next.

   (a) Set up a new github account (if you don't already have one) at `www.github.com`. You can also use the Queen Mary hosted git instance at `github.qmul.ac.uk` if you prefer for privacy.

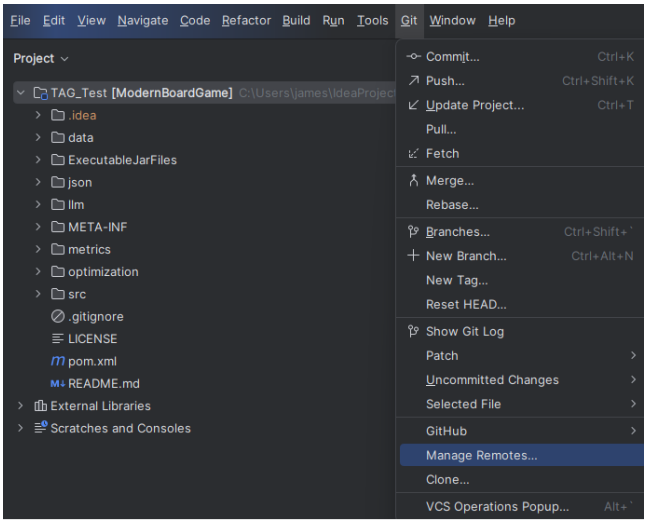
(b) Create a new repository. First click the green button on the top right of your main account page.



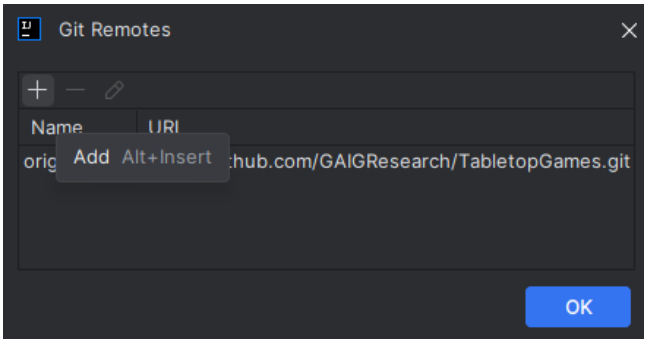
Then fill in the details with a new name for the repository and click, 'Create Repository'. Make a note of its URL, as we'll need that shortly.



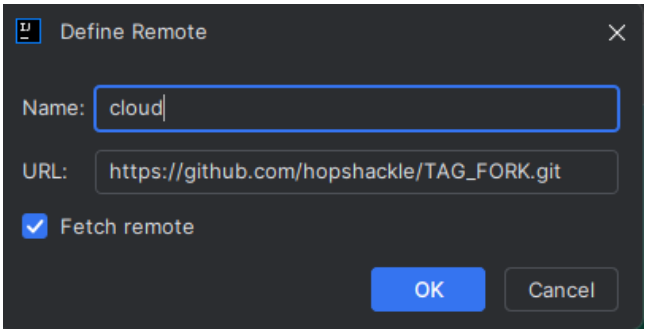
(c) Now we need to set up the remote link from the local repository to the new (empty) cloud one. Find the 'Git' → 'Manage Remotes' menu option:



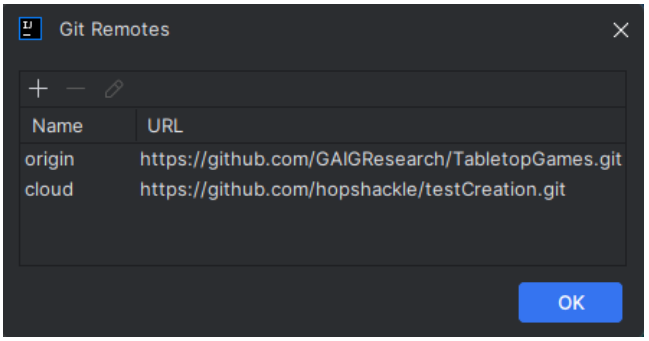
Then click the '+' button to add a new remote link. (At various points in later steps you will be prompted to login to your github account by IntelliJ to confirm access rights.)



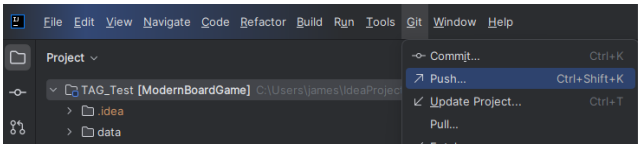
Fill in the URL of your new repository, and give it a name:



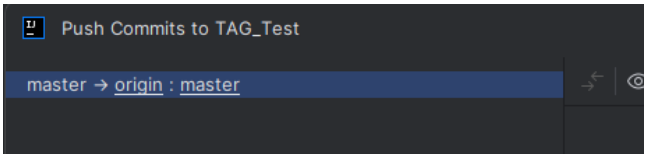
At which point you should have something like the list below. Your local repository is linked to the master TAG public repository as 'origin', and the new (still currently empty) fork as 'cloud' (or whatever you called it).



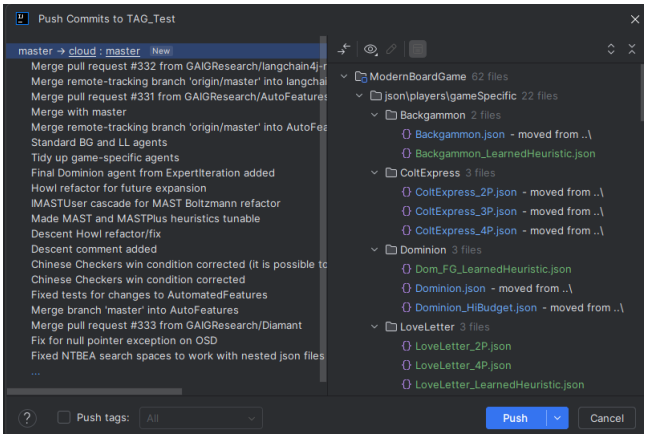
(d) Push the contents of your local repository to the new cloud one. Firstly find the 'Git' → 'Push' menu option:



Note that this defaults to push to 'origin', which is the main public repository, which is a bad idea, and anyway, you don;t have the access rights to do so. We therefore need to change this to push to 'cloud', by clicking on 'origin', and selecting the correct destination.



At which point you should have something like the screen below. Click on 'Push' and then go to github and confirm that all the code is now there too.



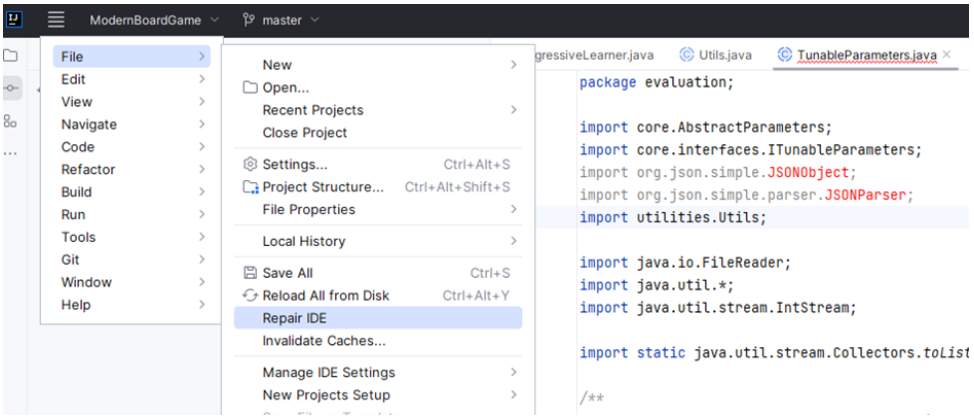
As you work on TAG the standard operations in the Git menu that are useful are:

- Pull. As the teaching team make changes (i.e. correct bugs) in the code, it is wise to always Pull in the latest changes from 'origin' each week.
- Push. Then push any changes you want to keep to 'cloud'.

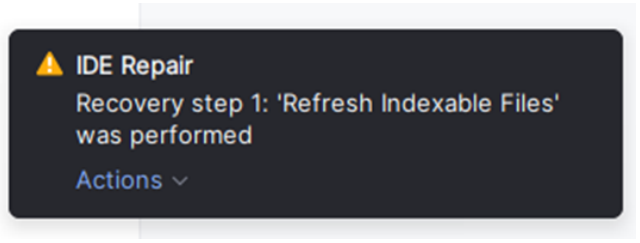
Troubleshooting with IntelliJ

Imports not found

When loading a new branch of TAG in IntelliJ, you may occasionally experience problems with some of the libraries not being loaded properly, which shows up as compilation errors. In the example below the `org.json.simple` library is not being found.

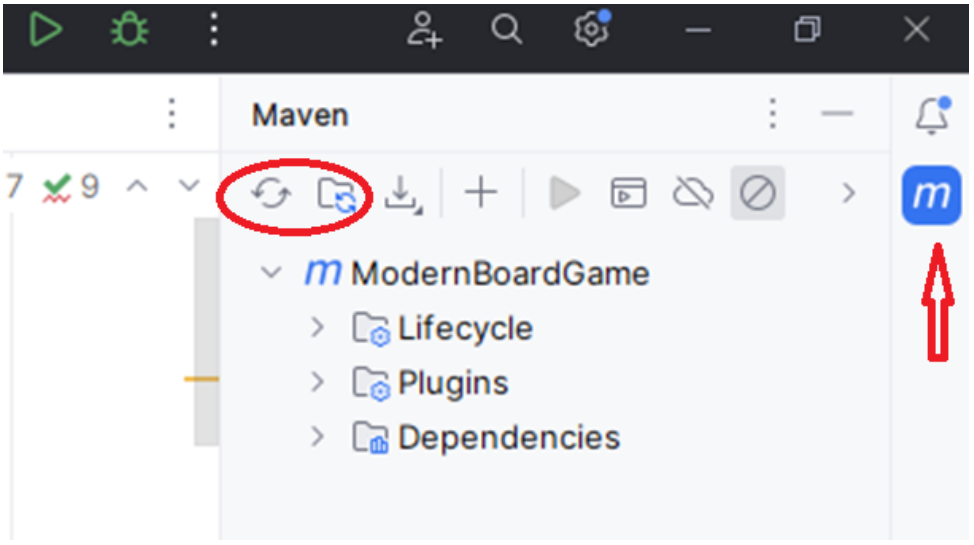


Usually this sort of problem can be fixed by using the File → Repair IDE menu option. Click this, and then work your way through the sequential actions until the issue is fixed. This will go through 6 or 7 steps in total, the first step is shown below. Then, in classic IT support fashion, ensure you reboot your computer.



Packages Missing

Another possible problem is that all the libraries have not yet been downloaded by Maven. If this is the first time you have loaded TAG into IntelliJ, then this will take a little time, so wait for a bit (say, 10 minutes. . . you should also see a background process running to load in the required libraries). In the case of this still not happening, open the Maven panel with the 'M' in the top right (the red circle).



Then try 'Reload' first (the first button on the left with a red arrow. And then, if that doesn't work, the next button for 'Generate Sources and Update Folders for all Projects'.

1 Overview

The objective of this lab is to familiarise yourself with TAG and process the information that is given to you in the different methods and classes. You should make sure you can run TAG in your machine (See the Getting Started section) before continuing.

2 Framework Description

The Tabletop Games Framework (TAG)[2] promotes research into general AI in modern tabletop games, facilitating the implementation of new games and AI players, while providing analytics to capture the complexities of the challenges proposed. The following definitions will help you understand the rest of this document:

The **game state** contains all the data for the current game, but no game logic. We define an **action** as an independent unit of game logic that modifies a given game state towards a specific effect (e.g. player draws a card; player moves their pawn). These actions are executed by the game players and are subject to certain **rules**: units of game logic. Rules dictate how a given **game state** is modified, for instance, checking the end of game conditions and the turn order, or moving a card from the player's hand into a discard pile. Rules are implemented in code either within an action or the game **forward model**.

All tabletop games use **components** (game objects sharing certain properties), whose state is modified during the game. TAG includes several predefined components to ease the development of new games, such as tokens (a game piece of a particular type), dice (with N sides), cards (with text, images or numbers), counters (with a numerical value), grid and graph boards. Components can also be grouped into collections: an area groups components in a map structure in order to provide access to them using their unique IDs, while a deck is an ordered collection with specific interactions available (e.g. shuffle, draw, etc.). Both areas and decks are considered components themselves.

There is a more detailed description on the structure of a game in TAG (and a diagram) at `https://tabletopgames.ai/wiki/games/creating/first_steps`.

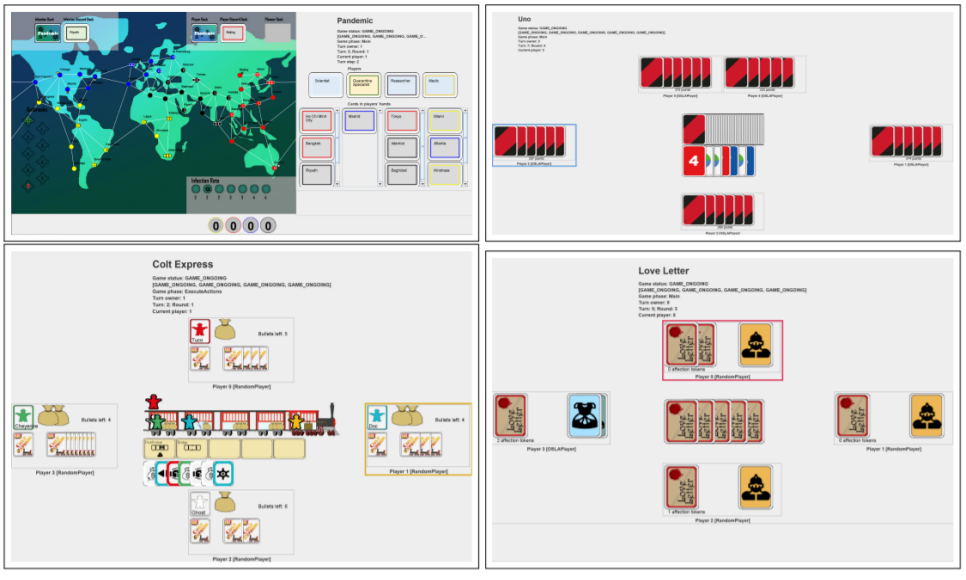


Figure 1: Four sample games implemented in the TAG framework - Pandemic, Uno, Lover Letter and Colt Express

Exercise 2 The first step is to inform yourself about the game - how it works, what kind of objects and actions are available, rules, etc. For this, you are recommended to first take a quick look at the documentation about the framework (introduction, game definition, framework structure, etc.) in the framework's Wiki:

`http://www.tabletopgames.ai/wiki/`

This Wiki includes definitions of terminology to better understand the framework, and information on games currently within it. The TAG wiki is generally a good place to find information about the framework. You can also find more information about TAG in this article published at EXAG 2020:

`https://rdgain.github.io/assets/pdf/papers/gaina2020tag.pdf`

3 Running the framework

There are different ways to run the framework and play yourself against the built-in AIs. We'll start with two simple ones: `core.Game.java` and `gui.Frontend.java`.

3.1 Game

In the `Game` class (found in the `core` package), you'll find the `main` method which can be run at the end of the file. Here, you can customise the way you run games by modifying some of the code. 4 steps are identified in the comments in this method:

3.1.1 Action Controller

The `Action Controller` is a Java object which is set up in the framework to capture user input in the Graphical User Interface (GUI). This needs only be created and attached to the game in order to work:

```
1 ActionController ac = new ActionController();
```

If this object is set to `null`, the game will run *without* visuals. However, there's a more intuitive way of setting the GUI on or off, the *useGUI* parameter.

Exercise 3 Locate this line at the beginning of the main() method in Game.java:

```
1 boolean useGUI = Utils.getArg(args, "gui", true);
```

Set the last value to *false*, which indicates that the GUI must run without visuals:

```
1 boolean useGUI = Utils.getArg(args, "gui", false);
```

Run the game now and verify that the no GUI is created. In the console message at the bottom of the IDE should indicate that the execution was run with no errors ("Process finished with exit code 0").

Finally, revert your change so GUI is enabled for the following exercises.

3.1.2 Game Seed

Next, the random seed for the run is initialised. This random seed is used for the random number generator needed to produce stochasticity in the games (for instance, to suffle cards randomly). Two games that are run using the same seed (e.g. `46748483L`) will produce the same sequence of random events. This is a `long` data type, and can take a specific value (to replicate the same results in different runs), or `System.currentTimeMillis()`, a Java method which returns a `long` value based on the current time (and therefore different for different runs).

```
1 long seed = System.currentTimeMillis();
```

3.1.3 Players

A few lines follow to initialise a list of **players**. This variable is a Java `ArrayList` data type, an ordered list. It needs to contain Java objects representing the players that will take part in the game. All objects added to this list need to extend the `AbstractPlayer` class, and the size of this list will indicate the number of players for the game. Multiple copies of the same player can be added. The following example would run a 3-player game with a human player, an AI player that plays at random, and an AI player controlled by the Monte Carlo Tree Search algorithm:

```
1 ArrayList<AbstractPlayer> players = new ArrayList<>();
2 players.add(new HumanGUIPlayer(ac));
3 players.add(new RandomPlayer());
4 players.add(new BasicMCTSPlayer());
```

Exercise 4 'Pandemic' is a game that can be played by maximum of 4 players. Modify the code in the main() function of the Game.java class to run a game of Pandemic with 4 players: a human GUI player, an OSLA Player, an MCTS player and a random player. Note that you may have to import some classes at the

top of Game.java for some of these objects (the IDE will give you a hint of what to import).

If the number of players indicated doesn't correspond with the number of players accepted by the game being run, an error will occur. The easiest way to check the number of players is in the class `games.GameType.java`. This class contains the descriptors of all the games in the framework which can be run, including:

- The number of players they accept (minimum and maximum).
- A list of categories the game belongs to.
- A list of mechanics the game uses.
- References to the classes which actually implement the game functionality and the specific ways needed to instantiate each game.

Exercise 5 Go to the class games.GameType.java (located at main/java/games/) and see if you can find the minimum and maximum number of players for the games:

- Tic Tac Toe,
- Connect4,
- Dots and Boxes,
- Pandemic,
- Poker.

The AI players which currently exist in the framework can be found in the `players` package, and can be instantiated for the players ArrayList as follows:

AI Player	Path	Java Construction Code	Opt. Constructor Arguments
FirstActionPlayer	players/simple/	`new FirstActionPlayer()`	-
Random	players/simple/	`new RandomPlayer ()`	`java.util.Random`
OSLA	players/simple/	`new OSLAPlayer ()`	`java.util.Random`
MaxNSearch	players/search/	`new MaxNSearchPlayer ()`	`MaxNSearchParameters`
RMHC	players/rmhc/	`new RMHCPlayer()`	random seed **or** `RMHCParams`
RHEA	players/rhea/	`new RHEAPlayer()`	random seed **or** `RHEAParams`
MCTS	players/basicMCTS/	`new BasicMCTSPlayer()`	random seed **or** `BasicMCTSParams`
HumanGUIPlayer	players/human/	`new HumanGUIPlayer(ac)`	`ActionController`
HumanConsolePlayer	players/human/	`new HumanConsolePlayer()`	-

A short description of each player is as follows (more details can be found at the end of this document):

- **FirstActionPlayer**: always chooses the first action in the list of available actions in a game state.
- **Random**: executes a random action at every step.
- **OSLA**: One Step Look Ahead. Looks one step into the future, by simulating the effect of all possible actions in the current game state and picks the one leading to the best next state (according to game-specific heuristic).
- MaxNSearch: This implements full (deterministic) tree search from the lectures, including $\alpha\beta$-pruning and Iterative Deepening as options.
- **MCTS**: Monte Carlo Tree Search[1]. Incrementally builds a game tree by simulating the effect of different actions starting from the current state up to 10 steps into the future, and picks the next immediate action that statistically leads to a better future state. Note that there are two versions of MCTS implemented in TAG: BasicMCTS and MCTS. For the purposes of this module, just use the BasicMCTSPlayer, which is designed to be simpler and easier to understand. (Feel free to look at the more sophisticated MCTSPlayer, but you **cannot** use this as a base agent for Assignment 1.)
- **RMHC**: Random Mutation Hill Climber[3]. Uses the simplest evolutionary algorithm to evolve a sequence of actions that leads to the best state 10 steps into the future, and picks the first action in the sequence.

- **RHEA**: Rolling Horizon Evolutionary Algorithm. This is a more sophisticated version of RMHC that introduces crossover and other evolution operators. It was the result of a Masters project, and you can find the detail in a paper on the 'Research' tab of the main website at `https://tabletopgames.ai/Research`.
- **HumanGUIPlayer**: allows a human to play in the Graphical User Interface (GUI).
- **HumanConsolePlayer**: allows a human to play via the console.

Most of these agents use heuristics to evaluate how good a game state (i.e. the current position of the player in the games, considering tokens, cards or other game elements) is. These heuristics are most often game-specific, but can also be custom. These are found in the package `players.heuristics`.

An example of a simple heuristic is the Score Heuristic (see players.heuristics.ScoreHeuristic.java). This heuristic (like others) evaluates a given game state in the function `evaluateState()`, which in this case looks like this:

```
public double evaluateState(AbstractGameState gs, int playerId) {
    double score = gs.getGameScore(playerId);
    if (gs.getPlayerResults()[playerId] == CoreConstants.GameResult.WIN)
        return score * 1.5;
    if (gs.getPlayerResults()[playerId] == CoreConstants.GameResult.LOSE)
        return score * 0.5;
    return score;
}
```

This method first retrieves the current score of the game from the game state that is meant to be evaluated (`gs`) for the player we are evaluating the position of (`playerId`):

```
double score = gs.getGameScore(playerId);
```

Then, it checks if the game has been already won or lost by the player, which can be found in the array returned by the game state function `getPlayerResults()`, in the position `[playerId]`. This array contains the game status for each player, hence the expression

```
gs.getPlayerResults()[playerId] == CoreConstants.GameResult.WIN
```

resolves to `true` if *playerId* already won. If the game is not over, the value of this state is the same as the score obtained. If the game is over, this function amplifies (or reduces) the value of the state by multiplying by 1.5 for a game won by *playerId* (or by 0.5 for a game lost by *playerId*, respectively).

Exercise 6 Inspect the classes in the package players.heuristics for the following Heuristics. Can you understand how do they work?

- Random Heuristic,
- Pure Score Heuristic,
- Win Only Heuristic,
- Leader Heuristic.

3.1.4 Run One

The last step is to run the game. Initially you can use the `RunOne` method that is being called in Game.java by default:

```
runOne(TicTacToe, players, seed, ac, randomizeParameters, listeners, turnPause);
```

This function takes the following arguments:

- Game to play: the data time of this argument is `GameType`, so you can check the class previously mentioned for the possible values (i.e. games available in the framework).
- Path to file with configuration for the run: it's possible to supply a file that contains the parameters for the run (e.g. players, seed, etc.). If this is `null`, parameters as specified in the code will be used.
- We know already the next 2 parameters: players and seed.
- `randomizeParameters`: if true, the parameters of the game will be randomised; if false, the game will run with default parameters. You can leave this value as `false` for now.

- `listeners`: game listeners can be attached to react to specific events or record data. More on this later.
- `ActionController`, as defined above.
- `turnPause`: indicate here the number of milliseconds that the GUI pauses between game turns.

There are several ways of indicating *which* game to run. The simplest one is to change the default value on the first line of the main() method:

```
1 //The String ``Pandemic'' is the default argument for the variable ``game''
2 String gameType = Utils.getArg(args, "game", "Pandemic");
```

If the framework is run without providing arguments (as you can run it now, by default), the default value of this (and subsequent) variables are being used.

Exercise 7 Modify the players list so two random players play a game, and make sure your GUI is enabled. Then, run the following games by modifying the default game to use. For this you have to indicate the following Strings (use the indicated exact capitalization):

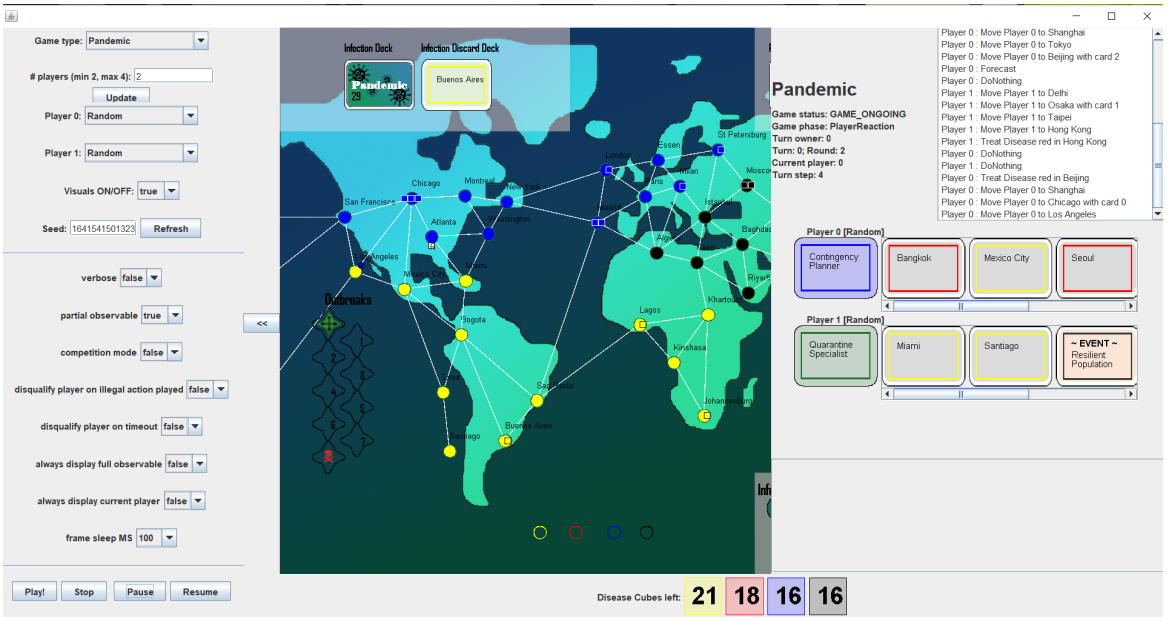
- "TicTacToe"
- "Connect4"
- "DotsAndBoxes"
- "ExplodingKittens"
- "Uno"
- "Poker"
- "Dominion"

Why do you think these games are already finished when you run then? You can change the value of the variable *turnPause* (for instance, to 100) to introduce a pause between turns and be able to see the moves players make.

3.2 Frontend

The second (and easier!) way to run the framework is through the `gui.Frontend.java` class. When run, this class produces a rough GUI which can be used to set different options and run games directly without the need to modify any code:

- Number of players for the run.
- For each player, select the player type (including human player options). Optionally, you can modify the player's parameters.
- Game to be run. Optionally, you can modify the game's parameters (if the game implements *tunable* parameters).
- Visuals can be toggled on or off.
- Set random seed.
- Set core parameters (including toggling partial observability and competition mode, setting display options or modifying the frame rate).
- Play, pause, resume and stop game buttons.



Note: depending on your screen size there may be resolution issues with running this option (which may prevent you from seeing all the options or even running games at all. In this case, please revert to the first option of running the framework, or try the `gui.FrontendSimple` class.

Exercise 8 Run FrontEnd, and try a few games. Experiment with different player numbers and setting different agents.

You will note that (with a few exceptions) the GUIs for the games are quite basic. This is quite intentional, and the main use of TAG as a research framework is to run large numbers of games without a GUI (in 'headless' mode). This is an area in which TAG excels, and we'll pick this up in the next lab.

3.3 When you are finished

After the lab: 1 Choose a game and play a couple of 2-player games, first you against the random player, then against MCTS (do not use the cooperative game "Pandemic" for this).

After the lab: 2 Read the EXAG 2020 TAG paper: `http://tabletopgames.ai/assets/pdf/gaina2020tag.pdf`.
Optionally, you may want to browse some of the other TAG papers at `http://tabletopgames.ai/research`

References

[1] Cameron Browne, Edward Powley, Daniel Whitehouse, Simon Lucas, Peter Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Pérez Liébana, Spyridon Samothrakis, and Simon Colton. A Survey of Monte Carlo Tree Search Methods. In *IEEE Trans. on Computational Intelligence and AI in Games*, volume 4, pages 1–43, 2014.

[2] Raluca D. Gaina, Martin Balla, Alexander Dockhorn, Raul Montoliu, and Diego Perez-Liebana. TAG: a Tabletop Games Framework. In *Proceedings of the AIIDE workshop on Experimental AI in Games*, 2020.

[3] Raluca D. Gaina, Sam Devlin, Simon M Lucas, and Diego Perez-Liebana. Rolling Horizon Evolutionary Algorithms for General Video Game Playing. *IEEE Transactions on Games*, 2021.